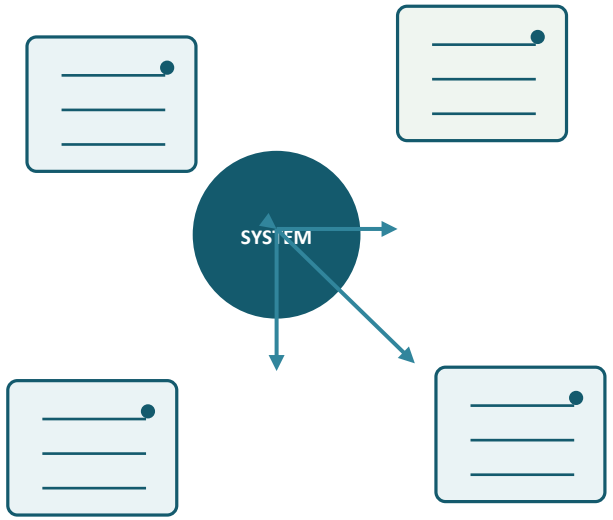
Design Challenges & Non-Functional Requirements of Distributed Systems



Non-Functional Requirements (NFR) - What makes a distributed system good?

NFRs - These are concerned with the quality of the system.

Certain requirements are common to many distributed systems.

We seek to understand the main design challenges and quality attributes that recur across distributed systems.

1 Resource Sharing

Share hardware, software and data

2 Openness

Integrate and evolve through well-defined interfaces

3 Concurrency

Coordinate simultaneous access to shared resources

4 Scalability

Handle growth without fundamental redesign

5 Fault Tolerance

Continue service despite component failures

6 Transparency

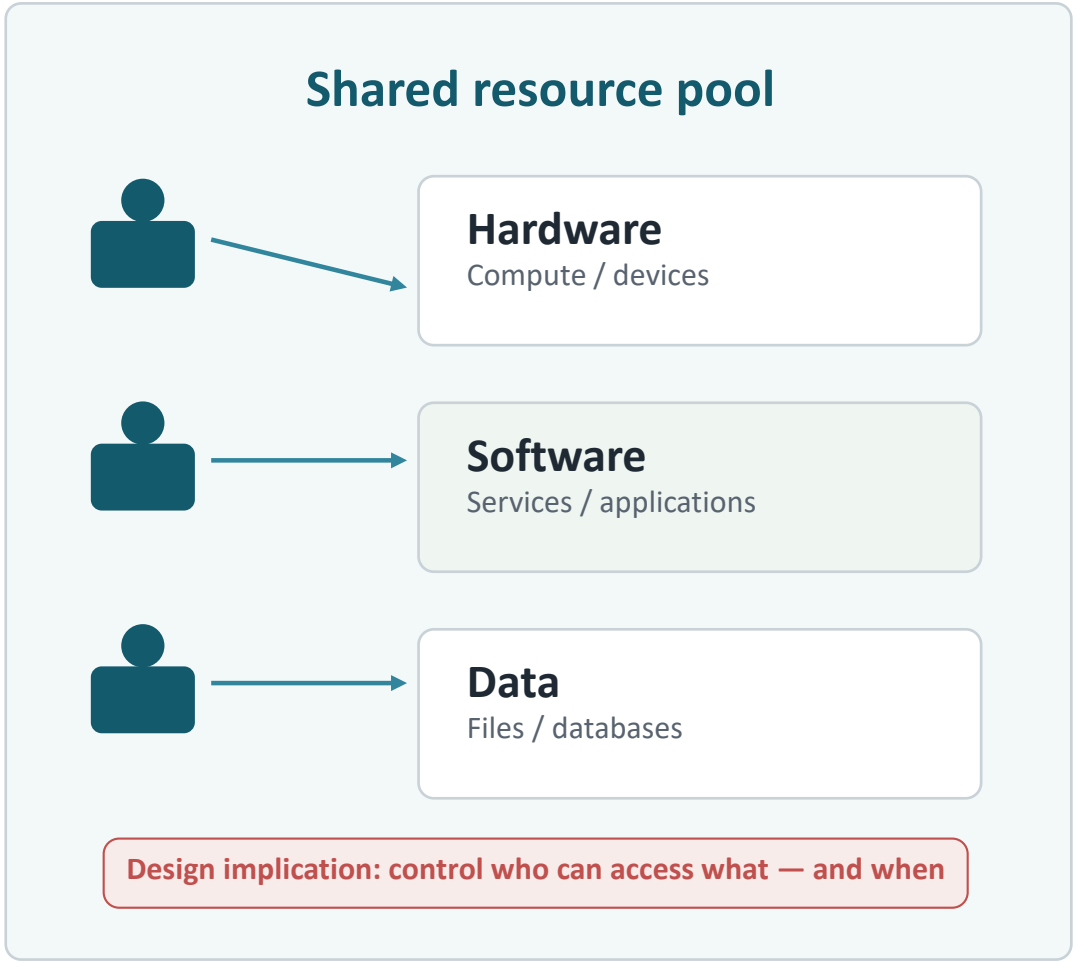
Hide unnecessary distribution complexity

Resource Sharing

DESIGN CHALLENGE 1

A distributed system allows hardware, software and data resources to be shared across the system.

- Cost effective for sharing expensive resources.
- Security implications due to many users accessing the common resources
  - How do control/restrict access? Resource manager components
  - Resource manager controls access, provides naming scheme and controls concurrency
  - Client-server Vs N-tier architecture (distributed objects)



Openness

Openness is concerned with extendibility and improvability of distributed systems

An open distributed system can be extended, integrated and improved without redesigning the whole system.

1

Published interfaces

Well defined and documented interfaces of components need to be published. Allows other components know what services are available

2

Recognised standards

System needs to adhere to recognized standards. Common protocols and formats support interoperability.

3

Replaceable components

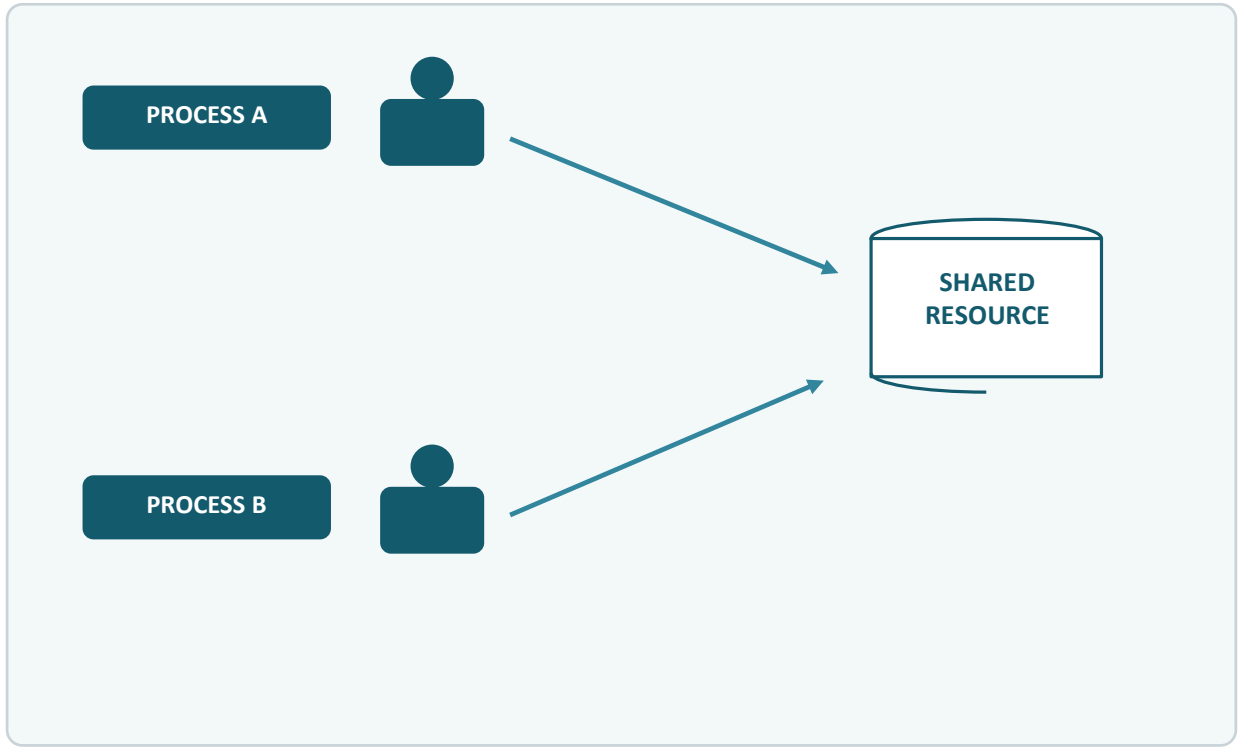
New or changed components can be integrated as requirements evolve.

Why it matters: supports organisational growth, preserves investment, and simplifies integration of new components.

Concurrency

Components in a distributed system execute concurrently — often accessing the same resources at the same time.

Components access and update shared resources (e.g. variables, databases, device drivers)



The risk

Integrity of the system may be violated if concurrent updates are not coordinated

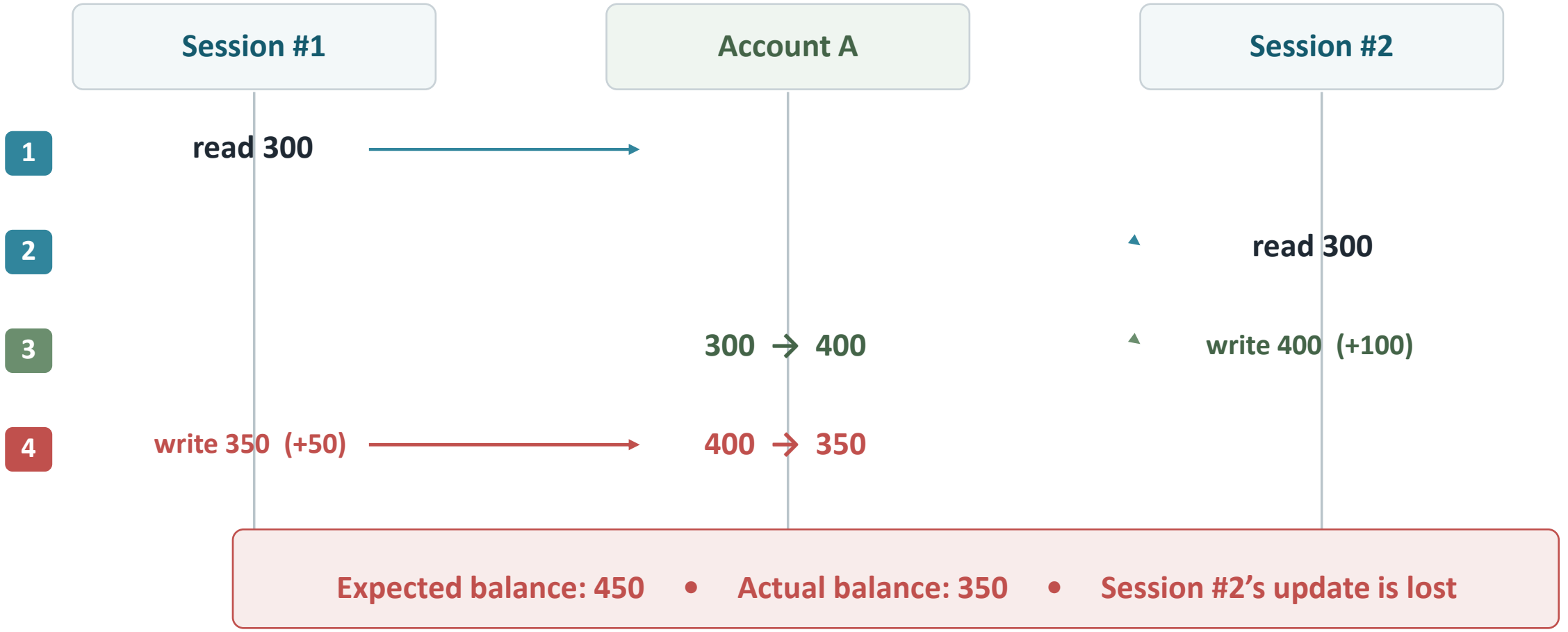
Lost update **Non-repeatable read**

Design question

How do we make concurrent access safe and predictable?

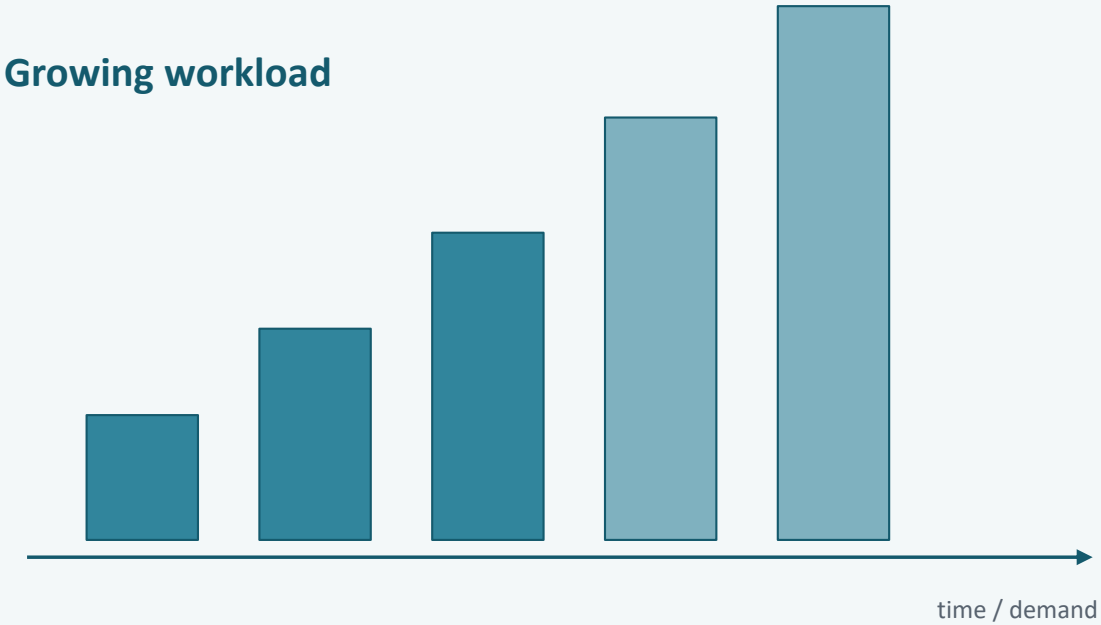
Concurrency: the Lost Update problem

Two sessions read the same value, update it independently, and one update is overwritten.



Scalability

A scalable distributed system can accommodate increasing workload without fundamental redesign.



The design should not need to be revisited simply because the system becomes larger.

Key ideas

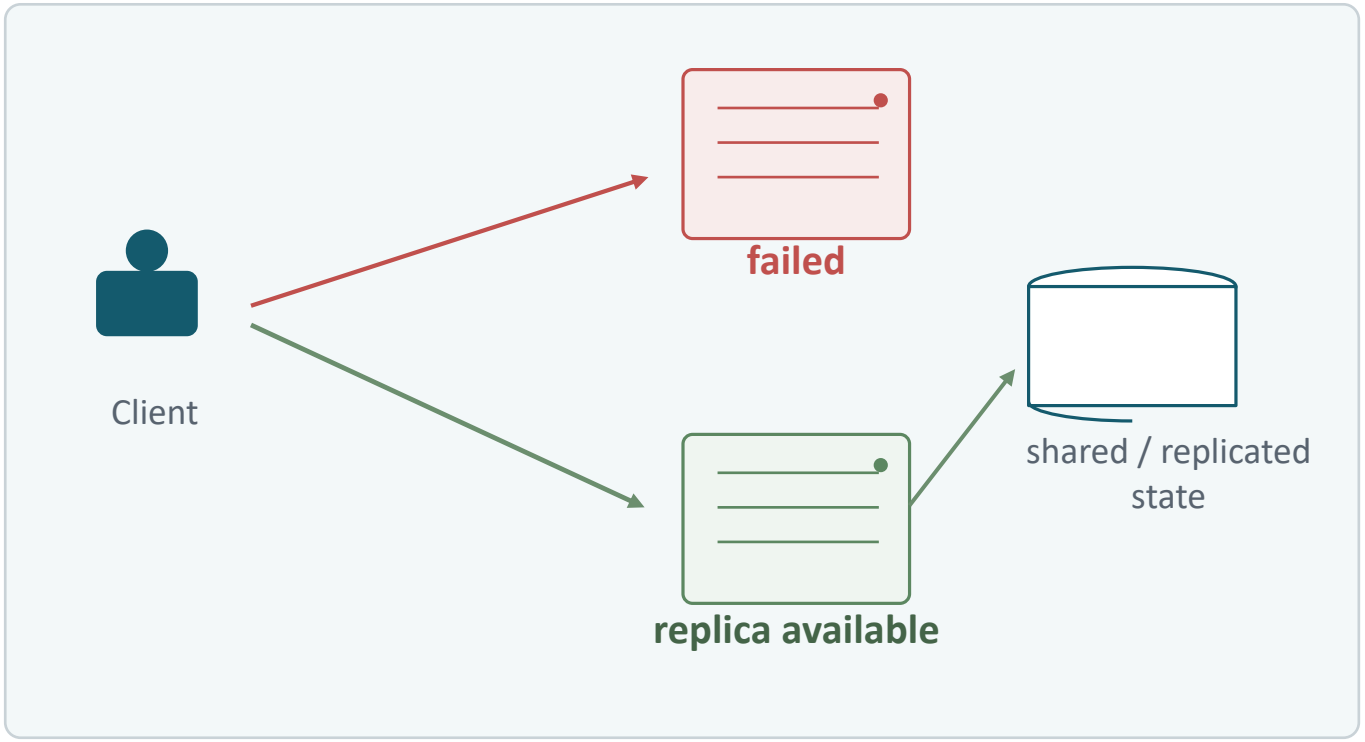
- Accommodate more users, requests or data.
- Improve or preserve response times as load increases.
- Often achieved by adding resources rather than redesigning component. Usually done by adding more and/or faster hosts or processors.
- Components should not need to be changed when scale of a system increases

Later: scale up/out • elasticity • Auto Scaling

Fault Tolerance

Hardware, software and networks fail. A fault-tolerant distributed system must maintain availability even at low levels of hardware/software/network reliability. Huge improvement over centralised systems

Operations that continue even in the presence of faults are referred to as fault-tolerant.



How it is commonly achieved

Redundancy **Replication**

Replicate components or data so another component can continue when one fails.

Important: distribution enables redundancy; it does not automatically guarantee availability.

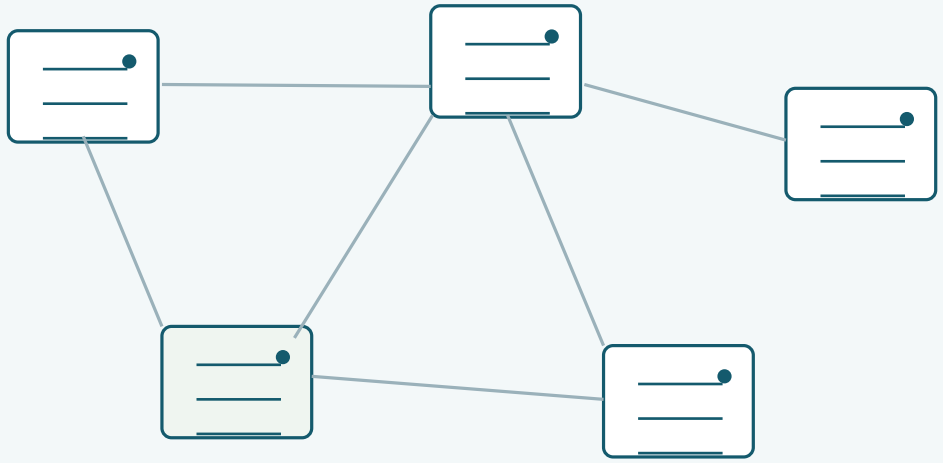
Transparency

Users and application developers should be able to perceive a distributed system as

- one coherent system.
- A collection of co-operating components which should be hidden.
- Efficient and cost-effective to construct and maintain (only possible if the complexity of distribution is hidden from the users).



what the user sees



many cooperating components

Transparency hides selected properties of distribution

Common Dimensions of Distribution Transparency

What aspect of distribution is hidden from the user or application?

Transparency = Hiding the complexity of <dimension>. There are eight dimensions of Transparency.

Access How it is accessed	**Location** Where it is	**Migration** That it moved	**Replication** That copies exist
Concurrency That others use it	**Scalability** That scale changed	**Performance** That reconfiguration occurred	**Failure** That a component failed

Transparency I: Access, Location & Migration

ACCESS

Hide differences in how local and remote resources are accessed. Enables local and remote components to be accessed using identical operations.

Example

- the same file operations can be used for local and remote files (e.g. NFS).
- Navigation of the web
- SQL Queries (standard).

It is critical in building distributed systems using heterogeneous computer architecture and programming languages.

LOCATION

Hide where a component or information object is physically located. Enables information objects to be local or remote without the user requiring knowledge of their location.

Example

- clients use a service name or endpoint without knowing the host location.
- Navigation of the web.
- Tables in a distributed database.

MIGRATION

Hide the fact that a component or information object has been relocated. A component can be relocated without users or clients noticing it.

Example

Allows the movement of information objects within a system without affecting the operations of users or application programs.

Transparency II: Replication & Concurrency

REPLICATION

A replica is a component copy that remains synchronized with its original.
Users and applications need not know that multiple synchronized copies of a component or data item exist.



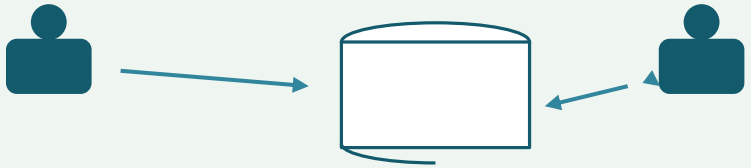
copies remain synchronized

Examples: distributed DBMS, replicated web/service instances

CONCURRENCY

Users and programmers are unaware that components request services concurrently.

Several processes can use shared information concurrently without interfering with one another.



shared data

Examples: DBMS transactions, shared file systems

Transparency III: Scalability, Performance & Failure

DIMENSIONS

SCALABILITY

The system can expand in scale without requiring changes to the system architecture or application algorithms.

E.g.: Internet, Distributed DB

What is hidden: changes in system scale.

PERFORMANCE

The system can be reconfigured as load varies in order to preserve or improve performance.

E.g.: Distributed make

What is hidden: performance-driven reconfiguration.

FAILURE

The system conceals selected component failures so users or applications can continue their tasks.
Often depends on concurrency and replication transparency

What is hidden: failure and recovery of components.

Recap: six questions to ask of a distributed system

Resource Sharing	What resources are shared — and who controls access?
Openness	Can components interoperate and evolve independently?
Concurrency	What happens when operations overlap?
Scalability	What happens as users, requests or data grow?
Fault Tolerance	What happens when a component fails?
Transparency	Which details of distribution should be hidden? Perceive distributed systems as One system. 8 Dimensions.